77. Сортировка массивов методом выбора

Сортировка массива — это процесс расположения всех элементов **массива** в определенном порядке. Очень часто это бывает полезным. Например, в вашем плейлисте аудио электронные песни отображаются в зависимости от времени получения, новые песни сначала были размещены, чем те, которые вы получили позднее, так, для или для между тем, когда вы переходите в свой плейлист, песни обычно находятся в информатике порядке, потому что так легче что-то найти. Все эти случаи включают в себя сортировку данных перед их дальнейшим выбором.

Оглавление:

1. Как работает сортировка?
2. Сортировка массивов методом выбора
3. Сортировка массивов методом выбора в C++
4. Функция std::sort()
5. Тест

Как работает сортировка?

Сортировка данных может сделать поиск внутри массива более эффективным не только для людей, но и для компьютеров. Например, рассмотрим случай, когда нам нужно учесть, отображается ли определенное имя в списке имен. Чтобы это учесть, нужно проверить каждый элемент массива на соответствие нашему значению. Поиск в массиве с множеством элементов может оказаться слишком неэффективным занятием.

Однако, гипотетически, что мы знаем, что наши данные отсортированы в алфавитном порядке. Тогда мы поиск начинается с первой буквы нашего значения и заканчивается буквой, которая идет следующей по алфавиту. В таком случае, если мы дошли до этой буквы, мы можем сделать что-то новое значит, что поиск в остальной части массива, так как в алфавитном порядке наш буквы мы уже прошли.

Не секрет, что есть алгоритмы поиска внутри отсортированных массивов и получше. Приведем простой алгоритм, мы можем искать отрицательный элемент в отсортированном массиве, содержащем 1 000 000 элементов, используя всего лишь 20 сравнений. Недостаточно, конечно же, является то, что сортировка массива с таким огромным количеством элементов — дело сравнительно затратное, и мы точно не выполнили бы даже одного полезного запроса.

В некоторых случаях сортировка массива делает поиск менее начучным. Например, мы ищем наилучший результат прохождения гонки среди судентов. Если массив не отсортирован, то нам придется пересмотреть каждый элемент массива, чтобы найти наивысший оценку. Если же массив отсортирован, то наивысшие оценки будет находиться либо на первой позиции, либо на последней в зависимости от метода сортировки массива в порядке возрастания или в порядке убывания, поэтому нам не нужно искать вообще!

Сортировка обычно выполняется путем повторного сравнения пар элементов массива с заменой значений, если они отвечают заданным критериям. Например, в которым это элементы сравниваются, зависит от того, какой алгоритм сортировки используется. Алгоритм сортировкиЗ, как будет показано ниже массив сортировки, в порядке возрастания или в порядке убывания.

Чтобы поменять два элемента местами, мы можем использовать **файлы** (переменные) из Стандартной библиотеки C++, которая определена в **заголовочном файле** algorithm. В C++11 функция std::swap() была перенесена в заголовочный файл utility.

```
#include <iostream>
#include <utility // для std::swap, с C++11 используем заголовок utility

int main()
{
    int a = 5;
    int b = 3;
    std::cout << "Before swap: a = " << a << " b = " << b << '\n';
    std::swap(a, b);
    std::cout << "After swap: a = " << a << " b = " << b << '\n';
```

Результат выполнения программы:

```
Before swap: a = 5, b = 3
After swap: a = 3, b = 3
```

После выполнения операции замены значений переменных a и b поменялись местами.

Сортировка массивов методом выбора

Существует множество способов сортировки массивов. Сортировка массивов методом выбора, пожалуй, самая простая для понимания, хотя и одна из самых медленных.

Для **сортировки массива методом выбора от наименьшего до наибольшего элемента** выполняется следующие шаги:

1. Начиная с элемента под индексом 0, ищем в массиве наименьшее значение.
2. Найденное значение меняем местами с нулевым элементом.
3. Повторяем шаги №1 и №2 для следующего индекса в массиве (отсортированный элемент больше не трогаем).

Другими словами, мы ищем наименьший элемент в массиве и перемещаем его на первое место. Затем ищем второй наименьший элемент и перемещаем его на второе место после первого наименьшего элемента. Этот процесс продолжается до тех пор, пока в массиве не закончатся неотсортированные элементы.

Вот пример работы этого алгоритма в массиве с 5 ю элементами:

{ 30, 50, 20, 10, 40 }

Сначала ищем наименьший элемент, начиная с индекса 0:

{ 30, 50, 20, 10, 40 }

Затем меняем местами наименьший элемент с элементом под индексом 0:

{ 10, 50, 20, 30, 40 }

Теперь, когда первый элемент массива отсортирован, мы его игнорируем. Ищем следующий наименьший элемент, но уже начиная с индекса 1:

{ 10, 50, 20, 30, 40 }

И меняем его местами с элементом под индексом 1:

{ 10, 20, 50, 30, 40 }

Теперь мы игнорируем первые два элемента. Ищем следующий наименьший элемент, начиная с индекса 2:

{ 10, 20, 50, 30, 40 }

И меняем его местами с элементом под индексом 2:

{ 10, 20, 30, 50, 40 }

Ищем следующий наименьший элемент, начиная с индекса 3:

{ 10, 20, 30, 50, 40 }

И меняем его местами с элементом под индексом 3:

{ 10, 20, 30, 40, 50 }

Ищем следующий наименьший элемент, начиная с индекса 4:

{ 10, 20, 30, 40, 50 }

И меняем его местами с элементом под индексом 4 (выполняется самосравнение, т.е. ничего не делаем):

{ 10, 20, 30, 40, 50 }

Готово!

{ 10, 20, 30, 40, 50 }

Обратите внимание, последнее сравнение всегда будет одиночным (т.е. самосравнение), что является лишней операцией, поэтому, фактически, мы можем остановить выполнение сортировки перед последним элементом массива.

Сортировка массивов методом выбора в C++

Вот как этот алгоритм реализован в C++:

```
int main()
{
    const int length = 5;
    int arr[length] { 30, 50, 20, 10, 40 };

    // Для хранения индекса начала цикла, наименьшего и индекса для обмена
    int startIndex, smallestIndex, temp;

    // Цикл по всем элементам массива, кроме последнего
    for (int startIndex = 0; startIndex < length - 1; ++startIndex)
    {
        // Предполагаем наименьшим первый элемент из оставшихся элементов
        smallestIndex = startIndex;

        // Цикл по всем элементам, начиная с startIndex + 1
        for (int currentIndex = startIndex + 1; currentIndex < length; ++currentIndex)
        {
            // Если текущий элемент меньше наименьшего элемента,
            if (arr[currentIndex] < arr[smallestIndex])
            {
                // запоминаем его индекс
                smallestIndex = currentIndex;
            }
        }

        // меняем местами наименьший элемент с startIndex
        temp = arr[startIndex];
        arr[startIndex] = arr[smallestIndex];
        arr[smallestIndex] = temp;
    }

    // Цикл по всем элементам массива для вывода его на экран
    for (int index = 0; index < length; ++index)
        std::cout << arr[index] << " ";

    return 0;
}
```

Наиболее значимой частью этого алгоритма является цикл внутри другого цикла (на наименьшей вложенный цикл). Внешний цикл (startIndex) перебирает элементы один за другим (индексацию). В каждой итерации внешнего цикла внутренний цикл (currentIndex) используется для поиска наименьшего элемента среди элементов, которые встречаются в массиве (начиная от startIndex + 1). smallestIndex отслеживает индекс наименьшего элемента, найденного внутренним циклом. Затем smallestIndex меняется значениями с startIndex, а значение startIndex увеличивается (чтобы переходит к следующему индексу массива, и процесс повторится.

Внимание! Если вы возможно проблемы с пониманием того, как работает программа, проведенная выше, то попробуйте запустить ее выполнение на листе бумаги вручную. Запишите начальные (неотсортированные) элементы массива горизонтально в строке в верхней части листа. Перрейте строки, ускользающие на то, каким элементам является startIndex, currentIndex и smallestIndex на данный момент. Прокрутите выполнение программы вручную и попробуйте строки на через изменения индексов. Поиск наименьшего элемента внешнего цикла наверху сверху, показывающие текущие состояние массива (фактические его элементы).

Сортировка massiva выполняется с помощью его алгоритма. Просто измените тип массива (int на std::string и инициализируйте его с помощью соответствующих значений.

Функция std::sort()

Поскольку операция сортировки массивов очень распространена, в Стандартной библиотеке C++ предусмотрен встроенная **функция сортировки std::sort()**. Она находится в заголовочном файле algorithm и выглядит следующим образом:

```
#include <algorithm
#include <iostream

int main()
{
    const int length = 5;
    int arr[length] { 30, 50, 20, 10, 40 };

    std::sort(arr, arr+length);

    for (int i = 0; i < length; ++i)
        std::cout << arr[i] << " ";

    return 0;
}
```

Тест

Задание №1

Напишите на листе бумаги выполнение сортировки следующего массива методом выбора (так, как мы это делали выше):

{30, 60, 20, 50, 40, 10}

Задание №2

Перепишите код программы из подготовки «Сортировка массивов методом выбора в C++» так, чтобы сортировка выполнялась в порядке убывания (от наибольшего числа к наименьшему). Если это может показаться сложным на первый взгляд, не не сами для или есть помочь.

Задание №3

Это задание уже немного сложнее.

Еще одним простым методом сортировки элементов является **«сортировка пузырьком»** или **«пузырьковая сортировка»**. Суть заключается в сравнении пары значений, которые находятся рядом, и, если сравниваемые одинаковые сортируют, значения из этой пары меняются местами. В таком образе элементы «всплывают» подобно пузырькам до конца массива. Хотя есть несколько способов сортировки «пузырьком», в этом задании мы будем придерживаться неоптимизированной версии, так как она отличается от остальных.

При неоптимизированной версии сортировки пузырьком выполняется следующие шаги для **сортировки массива от наименьшего до наибольшего значения**:

1. Сравниваются элемент массива под индексом 0 с элементом массива под индексом 1. Если элемент под индексом 0 больше элемента под индексом 1, то значения меняются местами.
2. Затем мы переходим к следующей паре элементов: элемент под индексом 1 и элемент под индексом 2 и так до тех пор, пока не достигнем конца массива.
3. Повторяем шаг №1 и шаг №2 до тех пор, пока весь массив не будет отсортирован.

Напишите программу, которая отсортирует следующий массив сортировкой пузырьком в соответствии с правилами, описанными выше:

```
int arr[5] = { 30, 50, 20, 10, 40 };
```

В конце программы выведите отсортированные элементы массива.

Подсказка: Если вы можете отсортировать один элемент за одну операцию, то это означает, что вам нужно будет повторить выполнение цикла столько раз, сколько есть начал в нашем массиве (по крайней мере, для гарантировать выполнение сортировки всего массива.

Задание №4

Рассмотрим следующую два решения оптимизации алгоритма сортировки пузырьком, который вы написали в предыдущем задании:

1. Обратите внимание, с каждым выполнением сортировки пузырьком наибольшие значения в массиве «пузырьков» до конца. После первой итерации последний элемент массива уже отсортирован. После второй итерации отсортированы предпоследний элемент массива и т.д. Следовательно, каждый последующий цикл не нужно перепроверять элементы, которые уже были отсортированы. Изменим цикл для соответствующим образом.

2. Если на протяжении всей итерации не выполнится ни одной замены, то мы знаем, что массив уже отсортирован. Внедрим проверку того, были ли сделаны какие-либо замены в текущей итерации, и, если нет — завершите выполнение цикла. Если дел был завершен, то выведите информацию о том, на какой итерации сортировка элементов завершилась.

Пример результата выполнения новой программы:

```
Early termination on iteration: 8
1 2 3 4 5 6 7 8 9
```

Ответ №1

```
30 60 20 50 40 10
30 60 20 50 40 10
10 20 60 50 40 30
10 20 30 50 60 40
10 20 30 40 50 60
10 20 30 40 50 60 (самозамена)
10 20 30 40 50 60 (самозамена)
```

Ответ №2

Просто измените:

```
if (arr[currentIndex] < arr[smallestIndex])
```

На:

```
if (arr[currentIndex] > arr[smallestIndex])
```

Также smallestIndex следует переименовать в largestIndex.

```
#include <iostream
#include <algorithm // для std::swap, с C++11 используем заголовок utility

int main()
{
    const int length{5};
    int arr[length] { 7, 5, 5, 4, 4, 5, 4, 2, 1, 3 };

    // Цикл по всем элементам массива, кроме последнего элемента (для выполнения)
    // Последний элемент не нужно сортировать
    for (int startIndex = 0; startIndex < length - 1; ++startIndex)
    {
        // Предполагаем первый элемент массива из оставшихся элементов (для выполнения)
        // Изначально это наибольшее значение в массиве
        int largestIndex = startIndex;
        int const numSwaps{0};

        // Цикл по всем элементам, начиная с startIndex + 1
        for (int currentIndex = startIndex + 1; currentIndex < length; ++currentIndex)
        {
            // Если текущий элемент больше наибольшего элемента,
            if (arr[currentIndex] > arr[largestIndex])
                // запоминаем его индекс
                largestIndex = currentIndex;
        }

        // Меняем местами наиб элемент массива с startIndex
        std::swap(arr[startIndex], arr[largestIndex]);
    }

    // Цикл по всем элементам массива для вывода его на экран
    for (int index = 0; index < length; ++index)
        std::cout << arr[index] << " ";

    return 0;
}
```

Ответ №3

```
#include <iostream
#include <algorithm // для std::swap, с C++11 используем заголовок utility

int main()
{
    const int length{5};
    int arr[length] { 7, 5, 5, 4, 4, 5, 4, 2, 1, 3 };

    // Цикл по всем элементам массива, кроме последнего элемента (для выполнения)
    // Последний элемент не нужно сортировать
    for (int startIndex = 0; startIndex < length - 1; ++startIndex)
    {
        // Предполагаем первый элемент массива из оставшихся элементов (для выполнения)
        // Изначально это наибольшее значение в массиве
        int largestIndex = startIndex;
        int const numSwaps{0};

        // Цикл по всем элементам, начиная с startIndex + 1
        for (int currentIndex = startIndex + 1; currentIndex < length; ++currentIndex)
        {
            // Если текущий элемент больше наибольшего элемента,
            if (arr[currentIndex] > arr[largestIndex])
                // запоминаем его индекс
                largestIndex = currentIndex;
        }

        // Меняем местами наиб элемент массива с startIndex
        std::swap(arr[startIndex], arr[largestIndex]);
    }

    // Цикл по всем элементам массива для вывода его на экран
    for (int index = 0; index < length; ++index)
        std::cout << arr[index] << " ";

    return 0;
}
```

Ответ №4

```
#include <iostream
#include <algorithm // для std::swap, с C++11 используем заголовок utility

int main()
{
    const int length{5};
    int arr[length] { 7, 5, 5, 4, 4, 5, 4, 2, 1, 3 };

    // Цикл по всем элементам массива, кроме последнего элемента (для выполнения)
    // Последний элемент не нужно сортировать
    for (int startIndex = 0; startIndex < length - 1; ++startIndex)
    {
        // Предполагаем первый элемент массива из оставшихся элементов (для выполнения)
        // Изначально это наибольшее значение в массиве
        int largestIndex = startIndex;
        int const numSwaps{0};

        // Цикл по всем элементам, начиная с startIndex + 1
        for (int currentIndex = startIndex + 1; currentIndex < length; ++currentIndex)
        {
            // Если текущий элемент больше наибольшего элемента,
            if (arr[currentIndex] > arr[largestIndex])
                // запоминаем его индекс
                largestIndex = currentIndex;
        }

        // Меняем местами наиб элемент массива с startIndex
        std::swap(arr[startIndex], arr[largestIndex]);
        swapped = true;
    }

    // Если цикл в этой итерации не выполнился ни одной замены, то мы знаем массив отсортирован
    if (swapped)
        break;

    // Если выполнение дошло до сюда, то мы знаем, что массив отсортирован и, следовательно, выводим результат
        std::cout << "Early termination on iteration: " << iteration + 1 << "\n";
        break;
    }

    // Цикл по всем элементам массива для вывода
    for (int index = 0; index < length; ++index)
        std::cout << arr[index] << " ";

    return 0;
}
```

Ответ №4

```
#include <iostream
#include <algorithm // для std::swap, с C++11 используем заголовок utility

int main()
{
    const int length{5};
    int arr[length] { 7, 5, 5, 4, 4, 5, 4, 2, 1, 3 };

    // Цикл по всем элементам массива, кроме последнего элемента (для выполнения)
    // Последний элемент не нужно сортировать
    for (int startIndex = 0; startIndex < length - 1; ++startIndex)
    {
        // Предполагаем первый элемент массива из оставшихся элементов (для выполнения)
        // Изначально это наибольшее значение в массиве
        int largestIndex = startIndex;
        int const numSwaps{0};

        // Цикл по всем элементам, начиная с startIndex + 1
        for (int currentIndex = startIndex + 1; currentIndex < length; ++currentIndex)
        {
            // Если текущий элемент больше наибольшего элемента,
            if (arr[currentIndex] > arr[largestIndex])
                // запоминаем его индекс
                largestIndex = currentIndex;
        }

        // Меняем местами наиб элемент массива с startIndex
        std::swap(arr[startIndex], arr[largestIndex]);
        swapped = true;
    }

    // Если цикл в этой итерации не выполнился ни одной замены, то мы знаем массив отсортирован
    if (swapped)
        break;

    // Если выполнение дошло до сюда, то мы знаем, что массив отсортирован и, следовательно, выводим результат
        std::cout << "Early termination on iteration: " << iteration + 1 << "\n";
        break;
    }

    // Цикл по всем элементам массива для вывода
    for (int index = 0; index < length; ++index)
        std::cout << arr[index] << " ";

    return 0;
}
```